

Présentation Projet Programmation Impérative

Percet Gauthier Nikita Volianskyi Mark Melkonian

- 1 Introduction
- 2 Architecture du code
- 3 Analyse des graphiques
- 4 Analyse
- 5 Conclusion

- Dans le cadre du projet de programmation impérative, nous avons réalisé une implémentation de l'algorithme des K plus proches voisins (KPPV).
- L'objectif de ce projet était de réaliser un programme complet permettant d'exécuter l'algorithme des KPPV avec deux types de structures de données.
- Nous avons utilisé pour cela le langage C, la bibliothèque MLV, Python et Bash.

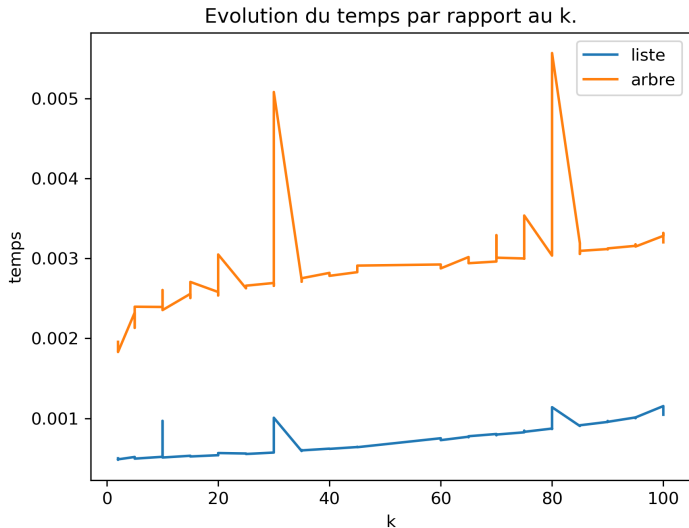
Structures de données

- **point.c** : Point multidimensionnel classifié.
- **vector.c** : Vecteur dynamique générique.
- **stack.c** : Pile générique.
- **point_distance.c** : Association point/distance.
- **ordered_points_list.c** : Liste ordonnée des k plus proches voisins.
- **points_list.c** : Liste de points (force brute).

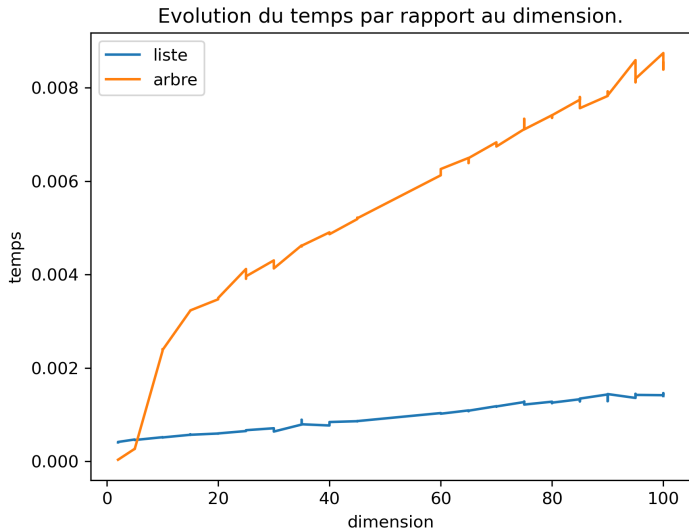
Algorithmes et interface

- **tree_kd.c** : Recherche KPPV par arbre KD.
- **save.c** : Lecture/écriture des données.
- **generate.c** : Génération de points aléatoires.
- **interface.c** : Affichage graphique (MLV).
- **main.c** : Point d'entrée et mesures de performance.
- **test.sh / graphique.py** : Tests et graphiques.

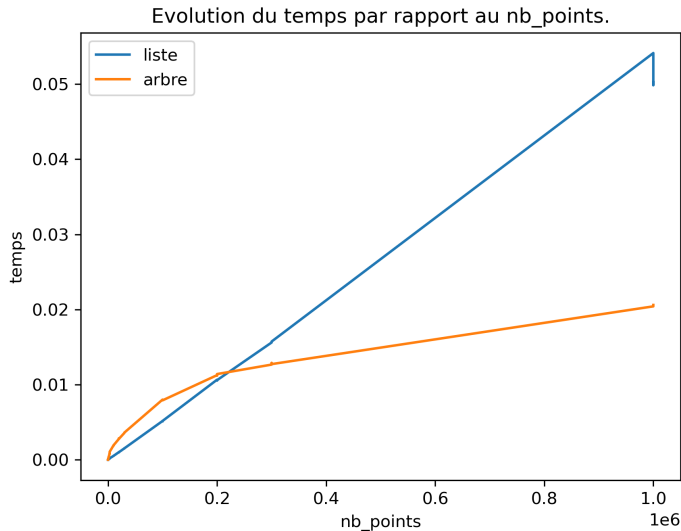
Variation de k (liste vs arbre)



Variation des dimensions (liste vs arbre)



Variation du nombre de points (liste vs arbre)



- variation de k voisins :
 - liste : évolution exponentielle
 - arbre : évolution logarithmique
- variation de dimension :
 - liste : évolution exponentielle
 - arbre : évolution logarithmique
- variation du nombre de point :
 - liste : évolution continue
 - arbre : évolution logarithmique

Conclusion

- Bilan des fonctionnalités implémentées.
- Difficultés rencontrées et solutions apportées.
- Perspectives d'amélioration.